

Deployment Guide

Wheat Water Productivity Dashboard

Ethiopian Institute of Agricultural Research · IWMI East Africa · FAO WaPOR Phase II
Generated 2026-08-19

Deployment — Railway

Staging first, then production. Both environments run the same image from the same `Dockerfile`; only environment variables differ.

What the image does

`Dockerfile` is a two-stage build:

1. `node:20-slim` installs the frontend dependencies and runs `vite build`.
2. `python:3.11-slim` installs the backend requirements, copies the built bundle to `frontend/dist`, and runs the notebook parity check.

The parity check (`tests/test_notebook_parity.py`) replays every result the IWMI reference notebook publishes through the shipped code and fails the build if any disagrees. It costs about a second and it is the reason an image whose estimates have drifted from the reference cannot reach a deployment.

There is nothing to train and no model artifact, so container boot is fast: the service loads no state beyond its own code.

The service runs as an unprivileged user and binds `$PORT`, which Railway injects.

Environment variables

VARIABLE	REQUIRED	PURPOSE
<code>PORT</code>	injected by Railway	Listen port
<code>WWP_PROVIDER</code>	yes, for real data	<code>synthetic</code> (default) or <code>wapor</code> . Left unset, the service serves demonstration figures — clearly labelled, but not real.
<code>WWP_CROP_PARAMS</code>	when EIAR parameters land	Absolute path to a JSON file overriding AOT, fc, mc and hi. Mount it as a volume or bake it in as <code>backend/crop_params.json</code> .
<code>WWP_WAPOR_LEVEL</code>	no	WaPOR product level. <code>L2</code> by default: the national 100 m products, which is what the reference implementation reads. <code>L1</code> is the 300 m global product; <code>L3</code> is the 20 m scheme mosaics and needs <code>WWP_WAPOR_SCHEME</code> .
<code>WWP_WAPOR_SCHEME</code>	for L3 only	Irrigation scheme code for the L3 mosaics: <code>KOG</code> (Koga) or <code>AWH</code> (Awash). Unset for L1 and L2.
<code>WWP_WAPOR_STORAGE</code>	no	Cloud storage root holding the dekadal GeoTIFFs, for a mirror.
<code>WWP_WAPOR_BASE</code>	no	Catalogue base URL, for a mirror or a pinned API version.
<code>WWP_CAMPAIGN_DATA</code>	no	Directory holding the 2026 campaign shapefiles, if they are on the host. Where they are found the Upload panel offers them as ready-made files; where they are not — the container image, and any public deployment — it offers a generated sample instead. The image deliberately does not carry them.
<code>WWP_CORS_ORIGINS</code>	only for cross-origin embedding	Comma-separated allowlist. The dashboard is served from the same origin, so this is unnecessary unless the EIAR site calls the API from another host.

Enabling real WaPOR retrieval

The default image serves synthetic data. Switching to live FAO WaPOR v3 needs two things:

1. `rasterio` in the image. It is commented out in `backend/requirements.txt` because GDAL is a large binary dependency that every deployment would otherwise pay for. Uncomment it and rebuild; the `python:3.11-slim` base carries manylinux wheels for it, so no extra apt packages are needed.
2. `WWP_PROVIDER=wapor`.

Then verify before trusting anything:

```
curl https://<domain>/api/wapor/check
```

That endpoint checks the catalogue is reachable, the NPP and AETI mapsets exist, their units are what the seasonal accumulation assumes, a scale factor is published, a concrete dekadal raster resolves to a download URL, and `rasterio` is importable. Every check must pass. The catalogue half of this contract has been verified against the live FAO service; the pixel read has not, so treat the first real analysis as something to sanity-check against known field values rather than as proven output.

Expect analyses to be considerably slower than on the synthetic provider: a five-month season is roughly 15 dekads, and the five-season trend multiplies that again. The 30-minute response cache absorbs repeats, but the first run over a new extent will take a while.

First deploy — staging

The Railway CLI needs an interactive browser login for `railway login`; the rest runs unattended.

Commands below are PowerShell (the project's development environment); bash equivalents follow in the next section.

```
npm install -g @railway/cli
railway login                # opens a browser

railway init                 # creates the project
railway environment new staging
railway environment staging
railway add --service wwp-dashboard

railway up --service wwp-dashboard --environment staging
railway domain               # mint a public URL
```

Setting an environment variable is the one place the two shells differ enough to trip you up:

	POWERSHELL	BASH / ZSH
Set a variable	<code>\$env:WWP_BASE = "https://host"</code>	<code>export WWP_BASE=https://host</code>
Read it back	<code>\$env:WWP_BASE</code>	<code>\$WWP_BASE</code>
Run two commands	<code>cd tests; node test_ui.mjs</code>	<code>cd tests && node test_ui.mjs</code>

PowerShell has no `export`, and **no spaces around the `=`**. Windows PowerShell 5.1 also has no `&&`: the bash snippets below chain with it, so substitute `;` there — `&&` is a parse error, not a warning.

Railway reads `railway.json`, which pins the Dockerfile builder, sets the health check to `/api/health` with a 300 s timeout, and **fixes the service at one replica**.

One replica is a correctness requirement, not a cost choice. The upload registry and the completed-run store are in-process dictionaries, so with two replicas a boundary uploaded through one instance is invisible to the other and CSV export links break at random. See the ToR Coverage document for the PostGIS work that lifts this constraint.

Verify staging

Point the test suites at the deployed URL — they need no local server:

```
export WWP_BASE=https://<your-staging-domain>
python tests/test_api_e2e.py          # 54 checks

cd tests && npm install playwright && npx playwright install chromium
node test_ui.mjs                      # 54 browser checks against the live site
```

The API suite covers all three AOI journeys, caching, idempotency, every validation path, and the WaPOR catalogue contract, so a green run against staging is a real smoke test rather than a liveness ping. It also asserts that the crop constant still matches the reference notebook, which catches a bad `WWP_CROP_PARAMS` in a deployed environment rather than in code review.

`GET /api/health` reports the active provider, and `GET /api/method` reports the equations and parameters actually in force. Check both after any environment change.

Promote to production

```
railway environment production
railway variables --set "WWP_PROVIDER=wapor"
railway up
railway domain
```

Then re-run `/api/wapor/check` and `/api/method` against the production domain before announcing it: the provider and the crop parameters are per-environment settings, and getting either wrong produces plausible-looking numbers rather than an error.

Before this is a public EIAR service

Beyond the environment setup above, the gaps listed in the ToR Coverage document still apply — chiefly enabling the WaPOR provider (every value served on the default configuration is synthetic), validating the crop parameters against EIAR field data, PostGIS for the upload and run stores, and Redis for the cache if the replica count ever rises above one.